

Exploring k-Nearest Neighbors in Recommendation Systems

Daizy Buluma

November 21, 2025

Abstract

1 Introduction

Today’s digital world is overflowing with information, making it nearly impossible for individuals to manually sift through the massive number of available options. As a result, recommender systems have become essential components of platforms such as Netflix, Amazon, Spotify, and TikTok. Their primary goal is to filter vast collections of items such as movies, products, songs, or videos, and present users with options they are most likely to enjoy. These systems work by analyzing users’ preferences, past behaviors, and item characteristics through interaction data such as impressions, clicks, ratings, likes, and purchases (Ghosh, 2024).

This paper focuses on the k-nearest neighbors (kNN) algorithm within the context of recommendation systems. Although kNN is one of the more straightforward algorithms in machine learning, it plays a foundational role in collaborative filtering methods. kNN identifies users or items that are most similar in behavior or preferences and bases predictions on these neighbors. Its simplicity and intuitive mechanics make it especially suitable for educational purposes and small scale recommendation tasks.

Before exploring the details of kNN, this paper outlines the broader landscape of recommendation systems to provide a foundation for understanding how kNN fits into the larger family of recommendation systems. The discussion then introduces and contrasts a different recommendation approach, Singular Value Decomposition (SVD) to highlight the strengths and weaknesses of kNN in practice.

Finally, the second half of the paper applies kNN to a real dataset to build and evaluate a simple interactive recommender system so as demonstrate how the method works in practice gauge its performance.

2 Overview of Recommender Systems

Recommendation systems work by estimating how much a user is likely to enjoy an item they have not yet interacted with. They do this by analyzing **user behavior**, such as past ratings or interactions and **item features**, such as genre, creator, or textual descriptions. The goal is to estimate a preference score, $r_{u,i}$, which measures how likely user u is to enjoy item i . Recommendations are then generated by selecting the items with the highest scores. To understand how k-nearest neighbors (kNN) fits into this broader landscape, it is useful to review two major families of recommendation approaches: **content-based approach** and **collaborative filtering** (There are hybrid recommenders too).

2.1 Content-Based Filtering

Content-based recommender systems generate suggestions by comparing the attributes of items a user previously liked to those of new items. These attributes might include genres, keywords, textual descriptions, or metadata. The item descriptions, which are labeled with ratings, are used as training data to create a user-specific classification or regression modeling problem. For each user, the training documents correspond to the descriptions of the items she has bought or rated (Aggarwal, 2016 p14). For example: If a shopper frequently purchases skincare products, the system recommends related cosmetic items.

The underlying assumption is that user preferences are largely consistent, and similar items will yield similar satisfaction (Ghosh, 2024). These models rely heavily on item features, making them effective in domains where rich metadata is available.

2.2 Collaborative Filtering

Collaborative filtering takes a different approach by focusing on patterns of behavior across users rather than item attributes. The key assumption is that users who behaved similarly in the past will behave similarly in the future. The main challenge in designing collaborative filtering methods is that the underlying ratings matrices are sparse (most unspecified). However, in collaborative filtering methods, these unspecified ratings can be imputed because the observed ratings are often highly correlated across various users and items. There are two main subtypes:

2.2.1 a) Memory-Based Methods

Memory-based methods make predictions directly from observed ratings by identifying similarities among users or items. kNN is the most common example and can be further categorized into **user-based kNN** and **item-based kNN**. The advantages of memory-based techniques are that they are simple to implement and the resulting recommendations are often easy to explain. On the other hand, memory-based algorithms do not work very well with sparse ratings matrices (Aggarwal, p. 14 2016).

2.2.2 b) Model-Based Methods

Model-based approaches use machine learning techniques to uncover latent patterns in the rating matrix. Examples include Matrix factorization and SVD and Neural networks. They tend to outperform memory-based methods on large, sparse datasets but are less transparent to users and developers. Notably, many hybrid recommenders combine kNN with model-based techniques, such as SVD, to balance simplicity and predictive power (Hssina, 2021).

3 K-Nearest Neighbors in Recommendation Systems

This section provides a deeper exposition of how kNN works, beginning with motivation and intuition, then moving into supervised-learning kNN, and finally examining how the algorithm is adapted for recommendation systems. A worked toy example concludes the section to illustrate the computations concretely.

3.1 Motivation: Why Neighbors Matter in Recommendations

The appeal of neighborhood-based methods lies in their simplicity as people with similar tastes tend to enjoy similar things (Nikolakopoulos 2021). If two users have consistently rated the same movies in comparable ways, then the films one user has enjoyed but the other has not yet seen become strong candidates for recommendation. This intuition, that preferences cluster and can be shared across neighbors, forms the foundation of k-Nearest Neighbors (kNN) in recommender systems. By exploiting these local patterns, kNN can generate useful predictions without the need for complex modeling.

3.2 How kNN Works in Supervised Learning

Before adapting k-nearest neighbors (kNN) to recommender systems, it helps to first understand how the algorithm works in its original setting. Unlike most algorithms we’ve seen before that estimate parameters or fit equations, kNN is a “lazy learner”. It stores the training data and defers computation until a query arrives. When a new example is presented, the algorithm measures its similarity to all stored examples, selects the k closest ones, and bases its prediction on their values.

In classification tasks, the new example is assigned to the majority class among its neighbors. In regression, the prediction is the average of their values. Weighted variants of kNN give greater influence to closer neighbors, which reduces the impact of outliers. The choice of k is critical to prevent over or under fitting. Too small a neighborhood makes the algorithm sensitive to noise, while too large a neighborhood can dilute meaningful signals. Because kNN does not assume any particular distribution of the data, it is considered non-parametric. This flexibility makes it easy to apply, but it also introduces challenges in high-dimensional spaces where distances lose contrast, a phenomenon known as the curse of dimensionality (Kramer, 2013, pg 19). (Still don't fully understand the phenomenon. will return to explain when I do)

3.3 How kNN Works in Recommender Systems

In recommendation settings, the task is to predict missing entries in a sparse user-item matrix of size (number of users) $\times$ (number of items). Each row represents a user, each column an item, and most cells are empty because users interact with only a small fraction of available items. An illustrative example is:

User / Item	A	B	C	D
U1	4	4	?	3
U2	4	5	2	?
U3	?	4	4	3
U4	2	?	5	?

Since most cells are empty, kNN adapts by computing similarities only on overlapping

ratings. This reliance on shared ratings is both a strength because it uses actual behavioral similarity and a weakness because sparsity may cause the similarities to be unreliable (Kramer). Two main variants are used:

- **User-based kNN:** The system identifies the k users most similar to the target user and infers preferences from their ratings. For example, if User 1 and User 2 have rated many of the same items similarly, then User 2's rating of a new item can help predict User 1's interest.
- **Item-based kNN:** Instead of comparing users, the system finds items that consistently receive similar ratings across users. If Item X and Item Y are rated alike by many users, then a user who liked Item X is predicted to enjoy Item Y as well. Item-based kNN is often more stable because item relationships remain consistent even as new users enter or leave the system. For this reason, modern commercial recommenders tend to use item-based kNN over user-based (Ricci et al. (2022)).

3.3.1 c) Similarity Metrics for Collaborative Filtering

In recommendation settings, not all distance metrics are equally effective. The most common are:

Cosine Similarity

Cosine similarity focuses on the angle between rating vectors rather than their magnitude:

$$\text{sim}(i, j) = \frac{\sum_u r_{u,i} r_{u,j}}{\sqrt{\sum_u r_{u,i}^2} \sqrt{\sum_u r_{u,j}^2}}$$

It is widely used because it remains stable even when users use different rating scales (Aggarwal, 2016).

Pearson Correlation

Pearson correlation adjusts for user biases:

$$\text{sim}(i, j) = \frac{\sum_u (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_u (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_u (r_{u,j} - \bar{r}_u)^2}}$$

This metric reduces problems with users who rate too generously or too strictly.

Adjusted Cosine

Used mostly in item-based kNN, this version subtracts user means before computing cosine:

$$\text{sim}(i, j) = \frac{\sum_u (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_u (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_u (r_{u,j} - \bar{r}_u)^2}}$$

It combines strengths of cosine and Pearson.

3.3.2 d) Prediction Formula

Once similarities have been computed, prediction is done using weighted averages.

User-Based kNN

For predicting user (u)'s rating for item (i):

$$\hat{r}_{u,i} = \frac{\sum_{v \in N_k(u)} \text{sim}(u, v) r_{v,i}}{\sum_{v \in N_k(u)} |\text{sim}(u, v)|}$$

Item-Based kNN

For predicting user (u)'s rating for item (i):

$$\hat{r}_{u,i} = \frac{\sum_{j \in N_k(i)} \text{sim}(i, j) r_{u,j}}{\sum_{j \in N_k(i)} |\text{sim}(i, j)|}$$

Both formulas emphasize closer neighbors more heavily.

3.3.3 c) Toy Example

We finish with toy examples adapted from Agrawal's book to emphasize idea:

Ratings:

User	A	B	C
U1	5	4	?
U2	4	4	2
U3	1	1	1

Goal: Predict U1's rating for Item C.

Step 1: Compute similarities

Compare Item A vs Item C using cosine similarity:

$$A = (4,1) \quad C = (2,1)$$

$$\text{sim}(A, C) \approx 0.98$$

Step 2: Weighted rating

U1 rated:

- A = 5
- B = 4

If A is most similar to C: $\hat{r}_{U1,C} \approx 5 \times 0.98 = 4.9$

Thus U1 is very likely to enjoy Item C.

3.4 Strengths and Weaknesses of kNN in Recommendation Systems

- Easy to Understand and Explain

- No Training Phase
- Works Well for Small or Dense Dataset

4 Comparison to a Model-Based Method (SVD)

- What SVD does (very high level)
- How it differs from kNN
- Strengths and weaknesses of both
- Maybe Table comparison ?
- When to choose kNN vs SVD
- Why SVD is dominant in large-scale systems

5 Application: Building a Simple Recommendation Engine with kNN

5.1 Dataset Description + Data Wrangling

5.2 Implementing kNN + Model Evaluation

5.3 Results + Interpretation

6 Reflection (?)

7 Conclusion

8 Appendix

References

Ricci, F., Rokach, L. & Shapira, B., eds (2022), *Recommender Systems Handbook*, 3 edn, Springer.

URL: <https://doi.org/10.1007/978-1-0716-2197-4>